

Índice

Informe de Revisión — Backend (api-integra-esavi)	1
1. Resumen	1
2. Mecanismo de carga	1
Orden de ejecución de seedData()	1
3. Archivos cargados en el primer arranque	2
3.1 Archivos CSV (upload_files/catalogos-csv/)	2
3.2 Archivos Excel (upload_files/catalogos-excel/)	2
4. Observaciones sobre los archivos y el proceso de carga	2
Críticas	2
Importantes	3
Menores / deuda técnica	4
5. Recomendaciones	5

Informe de Revisión — Backend (api-integra-esavi)

Fecha de revisión: 2026-06-11 **Componente revisado:** Carga inicial de catálogos mediante SeedService
Archivo principal: workspace/api-integra-esavi/src/integrator/service/seed.service.ts

1. Resumen

El backend carga los catálogos de homologación (CSV y Excel) mediante el servicio SeedService, que implementa OnApplicationBootstrap de NestJS. Esto significa que el método seedData() se ejecuta **automáticamente en cada arranque de la aplicación** (no únicamente la primera vez). La idempotencia se controla registro por registro con consultas findOne antes de cada save, por lo que en arranques posteriores no se duplican registros, pero el proceso completo vuelve a recorrer todos los archivos.

Cada paso de carga queda registrado en la tabla de procesos de sincronización (SyncProcess) mediante runSyncProcess(), con estados RUNNING / COMPLETED / FAILED.

2. Mecanismo de carga

Elemento	Detalle
Disparador	onApplicationBootstrap() → seedData() (seed.service.ts:68)
Disparo manual	POST /v1/seed expuesto en seed.controller.ts
Limpieza previa	cleanData() ejecuta TRUNCATE ... CASCADE sobre las tablas transaccionales TR_* y TC_GRUPO_ETARIO en cada arranque (seed.service.ts:149)
Auditoría	Cada bloque de carga se registra en SyncProcess vía runSyncProcess()
Idempotencia	findOne por claves de negocio antes de cada save

Orden de ejecución de seedData()

1. cleanData() — limpieza de tablas TR
2. seedTiposCatalogo() — tipos de catálogo (hardcodeados en el código)

3. seedCatalogos() — sexo, etnia, provincias VigiFlow, profesiones, unidades de edad, booleanos, roles (hardcodeados)
4. loadProvinciasFromCSV()
5. loadCantonesFromCSV()
6. loadParroquiasFromCSV()
7. loadIcd10meddraFromExcel()
8. loadSymptomToLltFromExcel()
9. loadWhodrugVacTempFromExcel()
10. loadWhodrugHomologacionVfFromExcel()
11. seedGruposEtarios() — grupos etarios (hardcodeados)

3. Archivos cargados en el primer arranque

Todos los archivos se leen desde `upload_files/` relativo a `process.cwd()`.

3.1 Archivos CSV (`upload_files/catalogos-csv/`)

Archivo	Destino	Formato esperado
<code>provincias_ecuador.csv</code>	TC_CATALOGO (tipo “Provincia”)	columnas: vigiflow, dhis2, homologada
<code>cantones_dhis2_ecuador.csv</code>	TC_CATALOGO (tipo “Cantón”)	columnas: vigiflow, dhis2, homologada
<code>parroquias_dhis2_ecuador.csv</code>	TC_CATALOGO (tipo “Parroquia”)	columnas: vigiflow, dhis2, homologada

3.2 Archivos Excel (`upload_files/catalogos-excel/`)

Archivo	Destino	Rango fijo de lectura
<code>ICD-10_to_MedDRA_28.0_Map-June-2025-ec10x</code>	<code>TC_CATALOGO</code>	A3:I11195 (~11.193 filas)
<code>Sintomas-DHIS2_MedDRA-LLT.xlsx</code>	<code>CtSymptom211t</code>	A2:E68
<code>20260126-WHODrug-vacunas-temporal.xlsx</code>	<code>WhodrugVacsTemp</code>	A2:T318
<code>20260126-ECU-WHODrug-Homologacion-Vigiflow-Vf.xlsx</code>	<code>WhodrugVfVigiflow</code>	A2:C36

4. Observaciones sobre los archivos y el proceso de carga

Críticas

#	Observación	Ubicación	Riesgo
O-01	cleanData() se ejecuta en cada arranque y hace TRUNCATE CASCADE de las tablas transaccionales (TR_PACIENTE, TR_NOTIFICACION, TR_DATOS_ESAVI, etc.). En producción esto borraría datos reales sincronizados cada vez que se reinicie el servicio.	seed.service.ts:72-77, 149-192	Alto — pérdida de datos
O-02	El SeedController expone POST /v1/seed, DELETE /v1/seed y DELETE /v1/seed/tr-tables (TRUNCATE de todas las tablas TR). Debe verificarse que estos endpoints estén protegidos (Keycloak/roles) o deshabilitados en producción.	seed.controller.ts	Alto — borrado vía API
O-03	Los errores de carga de CSV/Excel se capturan con try/catch interno que solo hace console.error sin relanzar; runSyncProcess registra el proceso como COMPLETED aunque el archivo haya fallado (p. ej. archivo ausente).	seed.service.ts:1043-1049, 1249-1251, etc.	Medio — falsos positivos en auditoría

Importantes

#	Observación	Ubicación	Riesgo
O-04	Los rangos de lectura de Excel están fijos en el código (A3:I11195, A2:E68, A2:T318, A2:C36). Si el archivo crece, las filas nuevas se ignoran silenciosamente; si decrece, se procesan filas vacías.	seed.service.ts:1194, 1264, 1324, 1431	Medio
O-05	Los nombres de archivo incluyen fecha/versión (20260126-..., ...June2025). Actualizar el catálogo requiere modificar y recompilar el código. Se sugiere externalizar nombres a configuración o leer el directorio.	seed.service.ts:1191, 1261, 1321, 1428	Medio
O-06	El parseo CSV usa split(',') simple: un valor con coma dentro de comillas rompe las columnas. Se recomienda una librería de CSV (p. ej. csv-parse).	seed.service.ts:1023, 1082, 1158	Medio
O-07	La carga de ICD-10 MedDRA (~11.000 filas) realiza un findOne + save por fila , sin lotes ni transacción. Arranque lento y posibilidad de carga parcial si el proceso se interrumpe.	seed.service.ts:1203-1244	Medio
O-08	SET session_replication_role = replica requiere privilegios de superusuario en PostgreSQL; con un usuario de aplicación estándar la limpieza fallará.	seed.service.ts:157, 4510	Medio

Menores / deuda técnica

#	Observación	Ubicación
O-09	TODOs pendientes en el propio código: “colocar auditoría correcta” y validación previa del archivo Excel (tipo de dato, filas/columnas vacías) antes de cargar, para evitar importar un catálogo equivocado.	seed.service.ts:1204, 1316, 1423
O-10	Expresiones redundantes col['A'] && col['A'] ? col['A'] : null (la condición se evalúa dos veces); además convierten valores legítimos falsy (0, '') en null.	seed.service.ts:1288-1292, 1350-1369
O-11	Existe un archivo temporal de Excel (~\$20260126-WH0Drug-vacunas-temporal.xlsx) en upload_files/catalogos-excel/ residuo de tener el archivo abierto en Excel. No afecta la carga (los nombres son exactos) pero no debe versionarse; agregar ~\$* y .DS_Store a .gitignore.	upload_files/catalogos-excel/
O-12	Cada arranque inserta 2 filas nuevas por paso en SyncProcess (RUNNING y COMPLETED); la tabla crece indefinidamente con los reinicios. Considerar actualizar el registro en lugar de insertar dos.	seed.service.ts:270-318
O-13	Código muerto comentado en volumen (seeders de datos ficticios, versión anterior de createSyncProcess). Conviene eliminarlo ahora que existe historial en git.	seed.service.ts:110-135, 322-348, 987-990
O-14	El CSV ECU-WH0Drug-Homologacion-Vacunas-VigiFlow.csv existe en catalogos-csv/ pero no es referenciado por ningún cargador (la versión usada es el Excel). Confirmar si es obsoleto.	upload_files/catalogos-csv/

5. Recomendaciones

1. **Condicionar** `cleanData()` a una variable de entorno (p. ej. `SEED_CLEAN_ON_BOOT=false` por defecto) o ejecutarla solo si las tablas están vacías.
2. **Proteger o deshabilitar** los endpoints de `SeedController` en ambientes productivos.

3. **Relanzar errores** de los cargadores de archivos para que SyncProcess refleje FAILED cuando un archivo falte o esté corrupto.
4. Detectar el rango de datos del Excel dinámicamente (`ws['!ref']`) en lugar de rangos fijos.
5. Externalizar rutas y nombres de archivos a configuración.
6. Cargar por lotes (`save([])` o `insert`) y dentro de una transacción los catálogos grandes (ICD-10 MedDRA).

Informe generado a partir de la revisión de `seed.service.ts` (1.539 líneas), `seed.controller.ts` y el contenido de `upload_files/`.